

Prashanth Terupelli Portfolio - Complete Code Documentation



Overview

This is a professional portfolio website built with React 19 + TypeScript + Tailwind CSS 4 on the frontend and Express + tRPC on the backend. It features a beautiful dark theme with cyan and purple accents, smooth animations, and optimized for Vercel deployment.

Live URL: <https://prashanth-portfolio.vercel.app>

Project Architecture

```
prashanth-portfolio/
├── client/ # Frontend (React + Vite)
│   ├── src/
│   │   ├── pages/
│   │   │   └── Home.tsx # Main portfolio page (ALL SECTIONS)
│   │   ├── components/ # Reusable UI components
│   │   ├── contexts/ # React contexts
│   │   ├── lib/
│   │   │   └── trpc.ts # tRPC client setup
│   │   ├── App.tsx # Main app component
│   │   ├── main.tsx # Entry point
│   │   └── index.css # Global styles & theme
│   ├── index.html # HTML template
│   └── public/ # Static assets
├── server/ # Backend (Express + tRPC)
│   ├── routers.ts # API endpoints
│   ├── email.ts # Email sending logic
│   ├── db.ts # Database queries
│   ├── _core/ # Core infrastructure
│   │   ├── index.ts # Server entry point
│   │   ├── env.ts # Environment variables
│   │   ├── context.ts # tRPC context
│   │   └── trpc.ts # tRPC setup
│   ├── storage.ts # S3 file storage
│   ├── drizzle/ # Database schema
│   │   └── schema.ts # Table definitions
│   └── shared/ # Shared types & constants
└─ ## File: client/src/pages/Home.tsx
   ``typescript
import { Button } from "@components/ui/button";
import { Mail, Github, Linkedin, Download, ExternalLink, ArrowRight } from
import { useEffect, useState } from "react";
import { trpc } from "@lib/trpc";
import { toast } from "sonner";

/**
 * Portfolio Design Philosophy: Data-Driven Professional
 * - Dark theme with cyan (#00d9ff) and purple (#7c3aed) accents
 * - Metric-focused achievements and quantifiable impact
 * - Smooth scroll animations and interactions
 * - Professional yet approachable aesthetic
```

```
*/
```

```
interface NavItem {  
  id: string;  
  label: string;  
  href: string;  
}
```

```
const navItems: NavItem[] = [  
  { id: "hero", label: "Home", href: "#hero" },  
  { id: "about", label: "About", href: "#about" },  
  { id: "skills", label: "Skills", href: "#skills" },  
  { id: "experience", label: "Experience", href: "#experience" },  
  { id: "contact", label: "Contact", href: "#contact" },  
];
```

```
interface Skill {  
  category: string;  
  items: string[];  
}
```

```
const skills: Skill[] = [  
  {  
    category: "Data Analytics & Technical",  
    items: ["SQL", "Python (Pandas, NumPy)", "Data Wrangling", "ETL Automati  
  },  
  {  
    category: "Analytics & Business",  
    items: ["Data Quality Analysis", "Root-Cause Analysis", "Stakeholder  
  },  
  {  
    category: "Tools & Platforms",  
    items: ["Snowflake", "Power BI", "Jira", "Confluence", "GraphQL"],  
  },  
];
```

```
interface Achievement {  
  metric: string;  
  description: string;  
  icon: string;  
}
```

```
const achievements: Achievement[] = [  
  {
```

```

    metric: "70%",
    description: "Data product performance boost through ETL automation",
    icon: "📈",
  },
  {
    metric: "60%+",
    description: "Validation efficiency improvement, scaled from 1 to 100%",
    icon: "✅",
  },
  {
    metric: "$96K",
    description: "Annual cost savings generated",
    icon: "💰",
  },
  {
    metric: "9%",
    description: "Reporting accuracy improvement through data analysis",
    icon: "🎯",
  },
];

```

```

interface ExperienceItem {
  title: string;
  company: string;
  period: string;
  highlights: string[];
}

```

```

const experience: ExperienceItem[] = [
  {
    title: "Senior Product Analyst (Data Analytics Focus)",
    company: "FactSet Research Systems",
    period: "Jun 2022 - Jan 2026",
    highlights: [
      "Boosted data product performance by 70% through ETL-driven automation",
      "Performed data wrangling by extracting, cleaning, transforming and loading data",
      "Automated Python and SQL pipelines, reducing manual effort by 40% through scripting",
      "Defined and tracked success metrics and KPIs to measure data product performance",
      "Analyzed large-scale datasets using Snowflake and SQL to identify trends and anomalies",
      "Conducted data validation and QA analysis, improving validation efficiency by 25%",
      "Built Power BI dashboards and Excel reports to visualize KPIs, trends, and forecasts",
    ],
  },
];

```

```

export default function Home() {
  const [activeSection, setActiveSection] = useState("hero");
  const [scrolled, setScrolled] = useState(false);
  const [formData, setFormData] = useState({ name: "", email: "", message: "" });
  const [isSubmitting, setIsSubmitting] = useState(false);

  useEffect(() => {
    const handleScroll = () => {
      setScrolled(window.scrollY > 50);

      // Update active section based on scroll position
      for (const item of navItems) {
        const element = document.getElementById(item.id);
        if (element) {
          const rect = element.getBoundingClientRect();
          if (rect.top ≤ 100 && rect.bottom ≥ 100) {
            setActiveSection(item.id);
            break;
          }
        }
      }
    };

    window.addEventListener("scroll", handleScroll);
    return () => window.removeEventListener("scroll", handleScroll);
  }, []);

  const scrollToSection = (id: string) => {
    const element = document.getElementById(id);
    if (element) {
      element.scrollIntoView({ behavior: "smooth" });
    }
  };

  const sendMessageMutation = trpc.contact.sendMessage.useMutation();

  const handleSubmit = async () => {
    if (!formData.name || !formData.email || !formData.message) {
      toast.error("Please fill in all fields");
      return;
    }

    setIsSubmitting(true);
  };

```

```

try {
  const result = await sendMessageMutation.mutateAsync(formData);
  if (result.success) {
    toast.success("Message sent successfully! We'll get back to you soon");
    setFormData({ name: "", email: "", message: "" });
  } else {
    toast.error(result.message || "Failed to send message");
  }
} catch (error) {
  console.error("Error sending message:", error);
  toast.error("There was an error sending your message. Please try again");
} finally {
  setIsSubmitting(false);
}
};

return (
  <div className="min-h-screen bg-background text-foreground">
    {/* Navigation */}
    <nav
      className={`fixed top-0 left-0 right-0 z-50 transition-all duration-300
        scrolled ? "bg-background/80 backdrop-blur-md border-b border-border" : ""
      }`>
      <div className="container flex items-center justify-between h-16">
        {/* Logo */}
        <div className="flex items-center gap-2">
          
          <span className="font-bold text-lg bg-gradient-to-r from-cyan to-blue text-transparent bg-clip-text">
            Prashanth
          </span>
        </div>

        {/* Nav Links */}
        <div className="hidden md:flex items-center gap-8">
          {navItems.map((item) => (
            <button
              key={item.id}
              onClick={() => scrollToSection(item.id)}
              className={`text-sm font-medium transition-colors duration-300
                ${item.active ? "text-foreground" : "text-background"}
              `}
            >
              {item.label}
            </button>
          ))}
        </div>
      </div>
    </nav>
  </div>
);

```

```

        activeSection ≡ item.id ? "text-cyan-400" : "text-mute
      }` }
    >
      {item.label}
    </button>
  )}}
</div>

{ /* CTA Button */ }
<Button
  size="sm"
  className="bg-cyan-500 hover:bg-cyan-600 text-background font
  onClick={() => scrollToSection("contact")}
>
  Get in Touch
</Button>
</div>
</nav>

{ /* Hero Section */ }
<section
  id="hero"
  className="relative min-h-screen flex items-center justify-center
>
  { /* Background Image */ }
  <div
    className="absolute inset-0 z-0"
    style={{
      backgroundImage:
        "url('https://d2xsxph8kpxj0f.cloudfront.net/3105196637769861
      backgroundColor: "cover",
      backgroundPosition: "center",
      opacity: 0.3,
    }}
  />

  { /* Gradient Overlay */ }
  <div className="absolute inset-0 bg-gradient-to-b from-background,

  { /* Content */ }
  <div className="container relative z-20 text-center max-w-4xl mx-
    <div className="space-y-6 animate-fade-in">
      <div className="space-y-2">
        <p className="text-cyan-400 font-medium text-sm tracking-wid

```

```

<h1 className="text-5xl md:text-7xl font-bold leading-tight">
  Prashanth Terupelli
</h1>
<p className="text-2xl md:text-3xl bg-gradient-to-r from-cyan-400 to-cyan-600">
  Senior Product Analyst
</p>
</div>

<p className="text-lg text-muted-foreground max-w-2xl mx-auto">
  Data-driven insights that drive real business impact. 3.7+ ⭐
</p>

{/* Achievements Grid */}
<div className="grid grid-cols-2 md:grid-cols-4 gap-4 py-8">
  {achievements.map((achievement, idx) => (
    <div
      key={idx}
      className="bg-card/50 backdrop-blur-sm border border-border"
    >
      <div className="text-3xl mb-2">{achievement.icon}</div>
      <div className="text-2xl font-bold text-cyan-400">{achievement.title}</div>
      <p className="text-xs text-muted-foreground mt-2">{achievement.description}</p>
    </div>
  ))}
</div>

{/* CTA Buttons */}
<div className="flex flex-col sm:flex-row gap-4 justify-center">
  <Button
    size="lg"
    className="bg-cyan-500 hover:bg-cyan-600 text-background"
    onClick={() => scrollToSection("about")}
  >
    Explore My Work
    <ArrowRight className="ml-2 w-4 h-4 group-hover:translate-x-1" />
  </Button>
  <a
    href="/Prashanth_Terupelli_Resume.pdf"
    download
    className="inline-flex items-center justify-center px-6 py-3"
  >
    <Download className="w-4 h-4 mr-2" />
    Download CV
  </a>
</div>

```

```

        </div>
    </div>
</div>

{/* Scroll Indicator */}
<div className="absolute bottom-8 left-1/2 -translate-x-1/2 z-20">
    <div className="w-6 h-10 border-2 border-cyan-400 rounded-full">
        <div className="w-1 h-2 bg-cyan-400 rounded-full animate-pulse">
        </div>
    </div>
</div>
</section>

{/* About Section */}
<section id="about" className="py-20 bg-card/30">
    <div className="container max-w-4xl">
        <div className="space-y-12">
            <div className="text-center space-y-4">
                <h2 className="text-4xl md:text-5xl font-bold" style={{ font:
                <div className="w-16 h-1 bg-gradient-to-r from-cyan-400 to-
            </div>

            <div className="grid md:grid-cols-2 gap-12">
                <div className="space-y-6">
                    <div>
                        <h3 className="text-xl font-bold text-cyan-400 mb-3">Who
                        <p className="text-muted-foreground leading-relaxed">
                            I'm an analytical and results-driven Senior Product An
                        </p>
                    </div>
                </div>

                <div>
                    <h3 className="text-xl font-bold text-cyan-400 mb-3">My
                    <p className="text-muted-foreground leading-relaxed">
                        I believe in writing clean, maintainable code and fol
                    </p>
                </div>

                <div>
                    <h3 className="text-xl font-bold text-cyan-400 mb-3">My
                    <p className="text-muted-foreground leading-relaxed">
                        To continuously enhance my skills in data analytics,
                    </p>
                </div>
            </div>
        </div>
    </div>

```

```

<div className="space-y-4">
  <div className="bg-background/50 border border-border rounded
    <h4 className="font-bold text-cyan-400 mb-4">Quick Facts
    <ul className="space-y-3 text-sm">
      <li className="flex items-start gap-3">
        <span className="text-purple-500 mt-1">></span>
        <span><strong>Location:</strong> Hyderabad, Telangana</span>
      </li>
      <li className="flex items-start gap-3">
        <span className="text-purple-500 mt-1">></span>
        <span><strong>Experience:</strong> 3.7+ years in Data</span>
      </li>
      <li className="flex items-start gap-3">
        <span className="text-purple-500 mt-1">></span>
        <span><strong>Education:</strong> B.Tech in Computer</span>
      </li>
      <li className="flex items-start gap-3">
        <span className="text-purple-500 mt-1">></span>
        <span><strong>Status:</strong> Immediate Joiner</span>
      </li>
    </ul>
  </div>

  <div className="bg-background/50 border border-border rounded
    <h4 className="font-bold text-cyan-400 mb-4">Connect With Me
    <div className="flex gap-3">
      <a
        href="mailto:tprashanth2301@gmail.com"
        className="p-3 bg-cyan-500/10 hover:bg-cyan-500/20"
        title="Email"
      >
        <Mail className="w-5 h-5 text-cyan-400" />
      </a>
      <a
        href="https://linkedin.com"
        target="_blank"
        rel="noopener noreferrer"
        className="p-3 bg-cyan-500/10 hover:bg-cyan-500/20"
        title="LinkedIn"
      >
        <Linkedin className="w-5 h-5 text-cyan-400" />
      </a>
    </div>
  </div>

```

```

        href="https://github.com"
        target="_blank"
        rel="noopener noreferrer"
        className="p-3 bg-cyan-500/10 hover:bg-cyan-500/20"
        title="GitHub"
      >
        <Github className="w-5 h-5 text-cyan-400" />
      </a>
    </div>
  </div>
</div>
</div>
</div>
</div>
</div>
</div>
</section>

{/* Skills Section */}
<section id="skills" className="py-20">
  <div className="container max-w-4xl">
    <div className="space-y-12">
      <div className="text-center space-y-4">
        <h2 className="text-4xl md:text-5xl font-bold" style={{ font
        <div className="w-16 h-1 bg-gradient-to-r from-cyan-400 to-
        </div>

      <div className="grid md:grid-cols-3 gap-6">
        {skills.map((skillGroup, idx) => (
          <div
            key={idx}
            className="bg-card/50 border border-border rounded-xl p
          >
            <h3 className="font-bold text-lg mb-4 text-cyan-400 gro
              {skillGroup.category}
            </h3>
            <div className="flex flex-wrap gap-2">
              {skillGroup.items.map((skill, sIdx) => (
                <span
                  key={sIdx}
                  className="px-3 py-1 bg-background border border-l
                >
                  {skill}
                </span>
              ))}
            </div>
          </div>
        ))}
      </div>
    </div>
  </div>

```



```

        </div>
      </div>
    </div>
  </section>

  { /* Contact Section */ }
  <section id="contact" className="py-20">
    <div className="container max-w-4xl">
      <div className="grid md:grid-cols-2 gap-12 items-center">
        <div className="space-y-8">
          <div className="space-y-4">
            <h2 className="text-4xl md:text-5xl font-bold" style={{ fr
            <p className="text-muted-foreground text-lg">
              Have a project in mind or just want to say hi? Feel free
            </p>
          </div>
        </div>

        <div className="space-y-4">
          <div className="flex items-center gap-4 group">
            <div className="p-3 bg-cyan-500/10 border border-cyan-400"
              <Mail className="w-6 h-6 text-cyan-400" />
            </div>
            <div>
              <p className="text-sm text-muted-foreground">Email Me
              <a href="mailto:tprashanth2301@gmail.com" className="
                tprashanth2301@gmail.com
              </a>
            </div>
          </div>
        </div>

        <div className="flex items-center gap-4 group">
          <div className="p-3 bg-purple-500/10 border border-purple
            <Linkedin className="w-6 h-6 text-purple-400" />
          </div>
          <div>
            <p className="text-sm text-muted-foreground">Professi
            <a href="https://linkedin.com" target="_blank" rel="no
              linkedin.com/in/prashanth
            </a>
          </div>
        </div>
      </div>
    </div>
  </div>
</div>

```

```

<div className="bg-card/50 border border-border rounded-2xl p-4">
  <div className="space-y-4">
    <div className="space-y-2">
      <label className="text-sm font-medium">Full Name</label>
      <input
        type="text"
        placeholder="John Doe"
        className="w-full bg-background border border-border"
        value={formData.name}
        onChange={(e) => setFormData({ ...formData, name: e.target.value })}
      />
    </div>
    <div className="space-y-2">
      <label className="text-sm font-medium">Email Address</label>
      <input
        type="email"
        placeholder="john@example.com"
        className="w-full bg-background border border-border"
        value={formData.email}
        onChange={(e) => setFormData({ ...formData, email: e.target.value })}
      />
    </div>
    <div className="space-y-2">
      <label className="text-sm font-medium">Your Message</label>
      <textarea
        rows={4}
        placeholder="Tell me about your project..."
        className="w-full bg-background border border-border"
        value={formData.message}
        onChange={(e) => setFormData({ ...formData, message: e.target.value })}
      />
    </div>
  </div>
  <div>
    <Button
      onClick={handleSubmit}
      disabled={isSubmitting || !formData.name || !formData.email}
      className="w-full bg-cyan-500 hover:bg-cyan-600 text-background"
    >
      {isSubmitting ? "Sending..." : "Send Message"}
    </Button>
  </div>
</div>

<div className="flex justify-center gap-4">

```

```

    <a
      href="mailto:tprashanth2301@gmail.com"
      className="flex items-center gap-2 px-4 py-2 bg-background
    >
      <Mail className="w-4 h-4 text-cyan-400" />
      <span className="text-sm">Email</span>
    </a>
    <a
      href="https://linkedin.com"
      target="_blank"
      rel="noopener noreferrer"
      className="flex items-center gap-2 px-4 py-2 bg-background
    >
      <Linkedin className="w-4 h-4 text-cyan-400" />
      <span className="text-sm">LinkedIn</span>
    </a>
    <a
      href="https://github.com"
      target="_blank"
      rel="noopener noreferrer"
      className="flex items-center gap-2 px-4 py-2 bg-background
    >
      <Github className="w-4 h-4 text-cyan-400" />
      <span className="text-sm">GitHub</span>
    </a>
  </div>
</div>
</div>
</section>

{/* Footer */}
<footer className="border-t border-border py-8 bg-card/30">
  <div className="container text-center text-muted-foreground text-sm">
    <p>© 2026 Prashanth Terupelli. All rights reserved.</p>
    <p className="mt-2">Crafted with passion by Prashanth Terupelli.</p>
  </div>
</footer>

{/* Scroll to Top Button */}
{scrolled && (
  <button
    onClick={() => scrollToSection("hero")}
    className="fixed bottom-8 right-8 z-40 p-3 bg-cyan-500 hover:bg
  >

```

```
        <ArrowRight className="w-4 h-4 rotate-[-90deg]" />
      </button>
    )}
  </div>
);
}
```

File: client/src/App.tsx

```
import { Toaster } from "@components/ui/sonner";
import { TooltipProvider } from "@components/ui/tooltip";
import NotFound from "@pages/NotFound";
import { Route, Switch } from "wouter";
import ErrorBoundary from "../components/ErrorBoundary";
import { ThemeProvider } from "../contexts/ThemeContext";
import Home from "../pages/Home";

function Router() {
  // make sure to consider if you need authentication for certain routes
  return (
    <Switch>
      <Route path="/" component={Home} />
      <Route path="/404" component={NotFound} />
      {/* Final fallback route */}
      <Route component={NotFound} />
    </Switch>
  );
}

// NOTE: About Theme
// - First choose a default theme according to your design style (dark or
//   to keep consistent foreground/background color across components
// - If you want to make theme switchable, pass `switchable` ThemeProvider

function App() {
  return (
    <ErrorBoundary>
      <ThemeProvider
        defaultTheme="dark"
        // switchable
      >
        <TooltipProvider>
          <Toaster />
          <Router />
        </TooltipProvider>
      </ThemeProvider>
    </ErrorBoundary>
  );
}
```

```
export default App;
```

File: client/src/main.tsx

```
import { trpc } from "@lib/trpc";
import { UNAUTHED_ERR_MSG } from '@shared/const';
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import { httpBatchLink, TRPCClientError } from "@trpc/client";
import { createRoot } from "react-dom/client";
import superjson from "superjson";
import App from "./App";
import { getLoginUrl } from "./const";
import "./index.css";

const queryClient = new QueryClient();

const redirectToLoginIfUnauthorized = (error: unknown) => {
  if (!(error instanceof TRPCClientError)) return;
  if (typeof window === "undefined") return;

  const isUnauthorized = error.message === UNAUTHED_ERR_MSG;

  if (!isUnauthorized) return;

  window.location.href = getLoginUrl();
};

queryClient.getQueryCache().subscribe(event => {
  if (event.type === "updated" && event.action.type === "error") {
    const error = event.query.state.error;
    redirectToLoginIfUnauthorized(error);
    console.error("[API Query Error]", error);
  }
});

queryClient.getMutationCache().subscribe(event => {
  if (event.type === "updated" && event.action.type === "error") {
    const error = event.mutation.state.error;
    redirectToLoginIfUnauthorized(error);
    console.error("[API Mutation Error]", error);
  }
});

const trpcClient = trpc.createClient({
  links: [
```

```
httpBatchLink({
  url: "/api/trpc",
  transformer: superjson,
  fetch(input, init) {
    return globalThis.fetch(input, {
      ...(init ?? {}),
      credentials: "include",
    });
  },
}),
],
});

createRoot(document.getElementById("root")!).render(
  <trpc.Provider client={trpcClient} queryClient={queryClient}>
    <QueryClientProvider client={queryClient}>
      <App />
    </QueryClientProvider>
  </trpc.Provider>
);
```

File: client/src/index.css

```
@import "tailwindcss";
@import "tw-animate-css";

/* Import Google Fonts */
@import url('https://fonts.googleapis.com/css2?family=Poppins:wght@400;500');

@custom-variant dark (&:is(.dark *));

@theme inline {
  --radius-sm: calc(var(--radius) - 4px);
  --radius-md: calc(var(--radius) - 2px);
  --radius-lg: var(--radius);
  --radius-xl: calc(var(--radius) + 4px);
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --color-card: var(--card);
  --color-card-foreground: var(--card-foreground);
  --color-popover: var(--popover);
  --color-popover-foreground: var(--popover-foreground);
  --color-primary: var(--primary);
  --color-primary-foreground: var(--primary-foreground);
  --color-secondary: var(--secondary);
  --color-secondary-foreground: var(--secondary-foreground);
  --color-muted: var(--muted);
  --color-muted-foreground: var(--muted-foreground);
  --color-accent: var(--accent);
  --color-accent-foreground: var(--accent-foreground);
  --color-destructive: var(--destructive);
  --color-destructive-foreground: var(--destructive-foreground);
  --color-border: var(--border);
  --color-input: var(--input);
  --color-ring: var(--ring);
  --color-chart-1: var(--chart-1);
  --color-chart-2: var(--chart-2);
  --color-chart-3: var(--chart-3);
  --color-chart-4: var(--chart-4);
  --color-chart-5: var(--chart-5);
  --color-sidebar: var(--sidebar);
  --color-sidebar-foreground: var(--sidebar-foreground);
  --color-sidebar-primary: var(--sidebar-primary);
  --color-sidebar-primary-foreground: var(--sidebar-primary-foreground);
```

```
--color-sidebar-accent: var(--sidebar-accent);
--color-sidebar-accent-foreground: var(--sidebar-accent-foreground);
--color-sidebar-border: var(--sidebar-border);
--color-sidebar-ring: var(--sidebar-ring);
}
```

```
:root {
  /* Portfolio Brand Colors */
  --primary: #00d9ff;
  --primary-foreground: #0f1419;
  --accent: #7c3aed;
  --accent-foreground: #f5f5f5;
  --success: #10b981;
  --success-foreground: #f5f5f5;

  /* Dark Theme - Professional */
  --radius: 0.65rem;
  --background: #0f1419;
  --foreground: #f5f5f5;
  --card: #1a202c;
  --card-foreground: #f5f5f5;
  --popover: #1a202c;
  --popover-foreground: #f5f5f5;
  --secondary: #2d3748;
  --secondary-foreground: #a0aec0;
  --muted: #4a5568;
  --muted-foreground: #a0aec0;
  --destructive: #ef4444;
  --destructive-foreground: #f5f5f5;
  --border: #2d3748;
  --input: #2d3748;
  --ring: #00d9ff;
  --sidebar: #1a202c;
  --sidebar-foreground: #f5f5f5;
  --sidebar-accent: #00d9ff;
  --sidebar-accent-foreground: #0f1419;
  --sidebar-border: #2d3748;
  --sidebar-ring: #00d9ff;

  /* Chart colors */
  --chart-1: #00d9ff;
  --chart-2: #7c3aed;
  --chart-3: #10b981;
  --chart-4: #f59e0b;
}
```

```

--chart-5: #ef4444;
}

.dark {
  /* Dark theme is default for this portfolio */
}

@layer base {
  * {
    @apply border-border outline-ring/50;
  }

  body {
    @apply bg-background text-foreground;
    font-family: Inter, sans-serif;
  }

  h1, h2, h3, h4, h5, h6 {
    @apply font-bold;
    font-family: Poppins, sans-serif;
  }

  button:not(:disabled),
  [role="button"]:not([aria-disabled="true"]),
  [type="button"]:not(:disabled),
  [type="submit"]:not(:disabled),
  [type="reset"]:not(:disabled),
  a[href],
  select:not(:disabled),
  input[type="checkbox"]:not(:disabled),
  input[type="radio"]:not(:disabled) {
    @apply cursor-pointer;
  }
}

@layer components {
  /**
   * Custom container utility that centers content and adds responsive padding
   */
  .container {
    width: 100%;
    margin-left: auto;
    margin-right: auto;
    padding-left: 1rem;
  }
}

```

```
padding-right: 1rem;
}

.flex {
  min-height: 0;
  min-width: 0;
}

@media (min-width: 640px) {
  .container {
    padding-left: 1.5rem;
    padding-right: 1.5rem;
  }
}

@media (min-width: 1024px) {
  .container {
    padding-left: 2rem;
    padding-right: 2rem;
    max-width: 1280px;
  }
}

/* Typography System */
.font-display {
  font-family: Poppins, sans-serif;
  @apply font-bold;
}

.font-body {
  font-family: Inter, sans-serif;
  @apply font-normal;
}

.font-mono {
  font-family: 'Fira Code', monospace;
}

/* Gradient Accent */
.gradient-accent {
  @apply bg-gradient-to-r from-cyan-400 to-purple-500 bg-clip-text text-
}

/* Glow Effect */
```

```

.glow-cyan {
  @apply shadow-lg shadow-cyan-500/20;
}

.glow-purple {
  @apply shadow-lg shadow-purple-500/20;
}
} ````

## File: client/index.html
````typescript
<!doctype html>
<html lang="en">

<head>
 <meta charset="UTF-8" />
 <meta
 name="viewport"
 content="width=device-width, initial-scale=1.0, maximum-scale=1" />
 <title>Prashanth Terupelli | Senior Product Analyst Portfolio</title>
 <!-- THIS IS THE START OF A COMMENT BLOCK, BLOCK TO BE DELETED: Google
 <link rel="preconnect" href="https://fonts.googleapis.com" />
 <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
 <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800;900" rel="stylesheet" />
 THIS IS THE END OF A COMMENT BLOCK, BLOCK TO BE DELETED -->
</head>

<body>
 <div id="root"></div>
 <script type="module" src="/src/main.tsx"></script>
 <script
 defer
 src="%VITE_ANALYTICS_ENDPOINT%/umami"
 data-website-id="%VITE_ANALYTICS_WEBSITE_ID%"></script>
</body>

</html>

```

## File: server/routers.ts

```
import { COOKIE_NAME } from "@shared/const";
import { getSessionCookieOptions } from "../_core/cookies";
import { systemRouter } from "../_core/systemRouter";
import { publicProcedure, router } from "../_core/trpc";
import { z } from "zod";
import { sendContactFormEmail } from "../email";

export const appRouter = router({
 system: systemRouter,
 auth: router({
 me: publicProcedure.query((opts) => opts.ctx.user),
 logout: publicProcedure.mutation(({ ctx }) => {
 const cookieOptions = getSessionCookieOptions(ctx.req);
 ctx.res.clearCookie(COOKIE_NAME, { ...cookieOptions, maxAge: -1 });
 return {
 success: true,
 } as const;
 }),
 }),
 contact: router({
 sendMessage: publicProcedure
 .input(
 z.object({
 name: z.string().min(1, "Name is required"),
 email: z.string().email("Valid email is required"),
 message: z.string().min(1, "Message is required"),
 })
)
 .mutation(async ({ input }) => {
 try {
 await sendContactFormEmail({
 name: input.name,
 email: input.email,
 message: input.message,
 });

 return {
 success: true,
 message: "Message sent successfully! I'll get back to you soon",
 };
 } catch {
 // ...
 }
 })
 })
});
```

```
 } catch (error) {
 console.error("Error sending contact message:", error);
 return {
 success: false,
 message: "Failed to send message. Please try again.",
 };
 }
 }
},
});
});
});

export type AppRouter = typeof appRouter;
```

## File: server/email.ts

```
import nodemailer from "nodemailer";
import { ENV } from "../_core/env";

let transporter: ReturnType<typeof nodemailer.createTransport> | null = null;

function getTransporter() {
 if (!transporter) {
 transporter = nodemailer.createTransport({
 service: "gmail",
 auth: {
 user: ENV.gmailUser,
 pass: ENV.gmailAppPassword,
 },
 });
 }
 return transporter;
}

export async function sendEmail(options: {
 to: string;
 subject: string;
 html: string;
 text?: string;
}) {
 try {
 const transporter = getTransporter();
 const result = await transporter.sendMail({
 from: ENV.gmailUser,
 to: options.to,
 subject: options.subject,
 text: options.text || options.html,
 html: options.html,
 });
 return { success: true, messageId: result.messageId };
 } catch (error) {
 console.error("Error sending email:", error);
 throw error;
 }
}

export async function sendContactFormEmail(data: {
```

```

 name: string;
 email: string;
 message: string;
 }) {
 const html = `
 <div style="font-family: Arial, sans-serif; max-width: 600px; margin:
 <h2 style="color: #00d9ff;">New Portfolio Contact Form Submission</h2>
 <hr style="border: none; border-top: 1px solid #e0e0e0; margin: 20px 0 20px 0;">

 <div style="margin: 20px 0;">
 <p>Name: ${escapeHtml(data.name)}</p>
 <p>Email: ${escapeHtml(data.email)}</p>
 </div>

 <div style="margin: 20px 0; background-color: #f5f5f5; padding: 15px 0 15px 40px;">
 <p>Message:</p>
 <p style="white-space: pre-wrap; word-wrap: break-word;">${escapeHtml(data.message)}</p>
 </div>

 <hr style="border: none; border-top: 1px solid #e0e0e0; margin: 20px 0 20px 0;">
 <p style="color: #999; font-size: 12px;">This email was sent from your portfolio</p>
 </div>
 `;

 return sendEmail({
 to: ENV.gmailUser,
 subject: `New Contact: ${data.name}`,
 html,
 text: `Name: ${data.name}\nEmail: ${data.email}\n\nMessage:\n${data.message}`
 });
 }

function escapeHtml(text: string): string {
 const map: Record<string, string> = {
 "&": "&",
 "<": "<",
 ">": ">",
 "'": """,
 '"': "'",
 };
 return text.replace(/[&<>"']/g, (m) => map[m]);
}

```

## File: server/db.ts

```
import { eq } from "drizzle-orm";
import { drizzle } from "drizzle-orm/mysql2";
import { InsertUser, users } from "../drizzle/schema";
import { ENV } from '../_core/env';

let _db: ReturnType<typeof drizzle> | null = null;

// Lazily create the drizzle instance so local tooling can run without a db
export async function getDb() {
 if (!_db && process.env.DATABASE_URL) {
 try {
 _db = drizzle(process.env.DATABASE_URL);
 } catch (error) {
 console.warn("[Database] Failed to connect:", error);
 _db = null;
 }
 }
 return _db;
}

export async function upsertUser(user: InsertUser): Promise<void> {
 if (!user.openId) {
 throw new Error("User openId is required for upsert");
 }

 const db = await getDb();
 if (!db) {
 console.warn("[Database] Cannot upsert user: database not available");
 return;
 }

 try {
 const values: InsertUser = {
 openId: user.openId,
 };
 const updateSet: Record<string, unknown> = {};

 const textFields = ["name", "email", "loginMethod"] as const;
 type TextField = (typeof textFields)[number];

 const assignNullable = (field: TextField) => {
```

```

 const value = user[field];
 if (value === undefined) return;
 const normalized = value ?? null;
 values[field] = normalized;
 updateSet[field] = normalized;
 };

 textFields.forEach(assignNullable);

 if (user.lastSignedIn !== undefined) {
 values.lastSignedIn = user.lastSignedIn;
 updateSet.lastSignedIn = user.lastSignedIn;
 }
 if (user.role !== undefined) {
 values.role = user.role;
 updateSet.role = user.role;
 } else if (user.openId === ENV.ownerOpenId) {
 values.role = 'admin';
 updateSet.role = 'admin';
 }

 if (!values.lastSignedIn) {
 values.lastSignedIn = new Date();
 }

 if (Object.keys(updateSet).length === 0) {
 updateSet.lastSignedIn = new Date();
 }

 await db.insert(users).values(values).onDuplicateKeyUpdate({
 set: updateSet,
 });
} catch (error) {
 console.error("[Database] Failed to upsert user:", error);
 throw error;
}

export async function getUserByOpenId(openId: string) {
 const db = await getDb();
 if (!db) {
 console.warn("[Database] Cannot get user: database not available");
 return undefined;
 }
}

```

```
const result = await db.select().from(users).where(eq(users.openId, openId));

return result.length > 0 ? result[0] : undefined;
}

// TODO: add feature queries here as your schema grows.
```

## File: server/\_core/index.ts

```
import "dotenv/config";
import express from "express";
import { createServer } from "http";
import net from "net";
import { createExpressMiddleware } from "@trpc/server/adapters/express";
import { registerOAuthRoutes } from "../oauth";
import { registerStorageProxy } from "../storageProxy";
import { appRouter } from "../routers";
import { createContext } from "../context";
import { serveStatic, setupVite } from "../vite";

function isPortAvailable(port: number): Promise<boolean> {
 return new Promise(resolve => {
 const server = net.createServer();
 server.listen(port, () => {
 server.close(() => resolve(true));
 });
 server.on("error", () => resolve(false));
 });
}

async function findAvailablePort(startPort: number = 3000): Promise<number> {
 for (let port = startPort; port < startPort + 20; port++) {
 if (await isPortAvailable(port)) {
 return port;
 }
 }
 throw new Error(`No available port found starting from ${startPort}`);
}

async function startServer() {
 const app = express();
 const server = createServer(app);
 // Configure body parser with larger size limit for file uploads
 app.use(express.json({ limit: "50mb" }));
 app.use(express.urlencoded({ limit: "50mb", extended: true }));
 registerStorageProxy(app);
 registerOAuthRoutes(app);
 // tRPC API
 app.use(
 "/api/trpc",
```

```
 createExpressMiddleware({
 router: appRouter,
 createContext,
 })
);
 // development mode uses Vite, production mode uses static files
 if (process.env.NODE_ENV === "development") {
 await setupVite(app, server);
 } else {
 serveStatic(app);
 }

 const preferredPort = parseInt(process.env.PORT || "3000");
 const port = await findAvailablePort(preferredPort);

 if (port !== preferredPort) {
 console.log(`Port ${preferredPort} is busy, using port ${port} instead`);
 }

 server.listen(port, () => {
 console.log(`Server running on http://localhost:${port}/`);
 });
}

startServer().catch(console.error);
```

## File: server/\_core/env.ts

```
export const ENV = {
 appId: process.env.VITE_APP_ID ?? "",
 cookieSecret: process.env.JWT_SECRET ?? "",
 databaseUrl: process.env.DATABASE_URL ?? "",
 oAuthServerUrl: process.env.OAUTH_SERVER_URL ?? "",
 ownerOpenId: process.env.OWNER_OPEN_ID ?? "",
 isProduction: process.env.NODE_ENV === "production",
 forgeApiUrl: process.env.BUILT_IN_FORGE_API_URL ?? "",
 forgeApiKey: process.env.BUILT_IN_FORGE_API_KEY ?? "",
 gmailUser: process.env.GMAIL_USER ?? "",
 gmailAppPassword: process.env.GMAIL_APP_PASSWORD ?? "",
};
```

## File: server/\_core/context.ts

```
import type { CreateExpressContextOptions } from "@trpc/server/adapters/express";
import type { User } from "../../drizzle/schema";
import { sdk } from "../sdk";

export type TrpcContext = {
 req: CreateExpressContextOptions["req"];
 res: CreateExpressContextOptions["res"];
 user: User | null;
};

export async function createContext(
 opts: CreateExpressContextOptions
): Promise<TrpcContext> {
 let user: User | null = null;

 try {
 user = await sdk.authenticateRequest(opts.req);
 } catch (error) {
 // Authentication is optional for public procedures.
 user = null;
 }

 return {
 req: opts.req,
 res: opts.res,
 user,
 };
}
```

## File: server/\_core/trpc.ts

```
import { NOT_ADMIN_ERR_MSG, UNAUTHED_ERR_MSG } from '@shared/const';
import { initTRPC, TRPCError } from '@trpc/server';
import superjson from 'superjson';
import type { TrpcContext } from '../context';

const t = initTRPC.context<TrpcContext>().create({
 transformer: superjson,
});

export const router = t.router;
export const publicProcedure = t.procedure;

const requireUser = t.middleware(async opts => {
 const { ctx, next } = opts;

 if (!ctx.user) {
 throw new TRPCError({ code: 'UNAUTHORIZED', message: UNAUTHED_ERR_MSG });
 }

 return next({
 ctx: {
 ...ctx,
 user: ctx.user,
 },
 });
});

export const protectedProcedure = t.procedure.use(requireUser);

export const adminProcedure = t.procedure.use(
 t.middleware(async opts => {
 const { ctx, next } = opts;

 if (!ctx.user || ctx.user.role !== 'admin') {
 throw new TRPCError({ code: 'FORBIDDEN', message: NOT_ADMIN_ERR_MSG });
 }

 return next({
 ctx: {
 ...ctx,
 user: ctx.user,
 },
 });
 })
);
```

```
 },
 });
 },
);
```

## File: server/storage.ts

```
// Preconfigured storage helpers for Manus WebDev templates
// Uploads via Forge Server presigned URL to S3 (PUT direct).
// Downloads return /manus-storage/{key} paths served via 307 redirect.

import { ENV } from "../_core/env";

function getForgeConfig() {
 const forgeUrl = ENV.forgeApiUrl;
 const forgeKey = ENV.forgeApiKey;

 if (!forgeUrl || !forgeKey) {
 throw new Error(
 "Storage config missing: set BUILT_IN_FORGE_API_URL and BUILT_IN_FORGE_API_KEY"
);
 }

 return { forgeUrl: forgeUrl.replace(/\/+$/, ""), forgeKey };
}

function normalizeKey(relKey: string): string {
 return relKey.replace(/^\/+/, "");
}

function appendHashSuffix(relKey: string): string {
 const hash = crypto.randomUUID().replace(/-/g, "").slice(0, 8);
 const lastDot = relKey.lastIndexOf(".");
 if (lastDot === -1) return `${relKey}_${hash}`;
 return `${relKey.slice(0, lastDot)}_${hash}${relKey.slice(lastDot)}`;
}

export async function storagePut(
 relKey: string,
 data: Buffer | Uint8Array | string,
 contentType = "application/octet-stream",
): Promise<{ key: string; url: string }> {
 const { forgeUrl, forgeKey } = getForgeConfig();
 const key = appendHashSuffix(normalizeKey(relKey));

 // 1. Get presigned PUT URL from Forge
 const presignUrl = new URL("v1/storage/presign/put", forgeUrl + "/");
 presignUrl.searchParams.set("path", key);
```

```

const presignResp = await fetch(presignUrl, {
 headers: { Authorization: `Bearer ${forgeKey}` },
});

if (!presignResp.ok) {
 const msg = await presignResp.text().catch(() => presignResp.statusText);
 throw new Error(`Storage presign failed (${presignResp.status}): ${msg}`);
}

const { url: s3Url } = (await presignResp.json()) as { url: string };
if (!s3Url) throw new Error("Forge returned empty presign URL");

// 2. PUT file directly to S3
const blob =
 typeof data === "string"
 ? new Blob([data], { type: contentType })
 : new Blob([data as any], { type: contentType });

const uploadResp = await fetch(s3Url, {
 method: "PUT",
 headers: { "Content-Type": contentType },
 body: blob,
});

if (!uploadResp.ok) {
 throw new Error(`Storage upload to S3 failed (${uploadResp.status})`);
}

return { key, url: `/manus-storage/${key}` };
}

export async function storageGet(relKey: string): Promise<{ key: string; url: string }> {
 const key = normalizeKey(relKey);
 return { key, url: `/manus-storage/${key}` };
}

export async function storageGetSignedUrl(relKey: string): Promise<string> {
 const { forgeUrl, forgeKey } = getForgeConfig();
 const key = normalizeKey(relKey);

 const getUrl = new URL("v1/storage/presign/get", forgeUrl + "/");
 getUrl.searchParams.set("path", key);

```

```
const resp = await fetch(getUrl, {
 headers: { Authorization: `Bearer ${forgeKey}` },
});

if (!resp.ok) {
 const msg = await resp.text().catch(() => resp.statusText);
 throw new Error(`Storage signed URL failed (${resp.status}): ${msg}`)
}

const { url } = (await resp.json()) as { url: string };
return url;
}
```

## File: drizzle/schema.ts

```
import { int, mysqlEnum, mysqlTable, text, timestamp, varchar } from "drizzle-orm";

/**
 * Core user table backing auth flow.
 * Extend this file with additional tables as your product grows.
 * Columns use camelCase to match both database fields and generated types.
 */
export const users = mysqlTable("users", {
 /**
 * Surrogate primary key. Auto-incremented numeric value managed by the database.
 * Use this for relations between tables.
 */
 id: int("id").autoincrement().primaryKey(),
 /** Manus OAuth identifier (openId) returned from the OAuth callback. Unique per user. */
 openId: varchar("openId", { length: 64 }).notNull().unique(),
 name: text("name"),
 email: varchar("email", { length: 320 }),
 loginMethod: varchar("loginMethod", { length: 64 }),
 role: mysqlEnum("role", ["user", "admin"]).default("user").notNull(),
 createdAt: timestamp("createdAt").defaultNow().notNull(),
 updatedAt: timestamp("updatedAt").defaultNow().onUpdateNow().notNull(),
 lastSignedIn: timestamp("lastSignedIn").defaultNow().notNull(),
});

export type User = typeof users.$inferSelect;
export type InsertUser = typeof users.$inferInsert;

// TODO: Add your tables here

File: shared/const.ts
```typescript
export const COOKIE_NAME = "app_session_id";
export const ONE_YEAR_MS = 1000 * 60 * 60 * 24 * 365;
export const AXIOS_TIMEOUT_MS = 30_000;
export const UNAUTHED_ERR_MSG = 'Please login (10001)';
export const NOT_ADMIN_ERR_MSG = 'You do not have required permission (10002)';
```

File: package.json

```
{
  "name": "prashanth-portfolio",
  "version": "1.0.0",
  "type": "module",
  "license": "MIT",
  "scripts": {
    "dev": "NODE_ENV=development tsx watch server/_core/index.ts",
    "build": "vite build && esbuild server/_core/index.ts --platform=node",
    "start": "NODE_ENV=production node dist/index.js",
    "check": "tsc --noEmit",
    "format": "prettier --write .",
    "test": "vitest run",
    "db:push": "drizzle-kit generate && drizzle-kit migrate"
  },
  "dependencies": {
    "@aws-sdk/client-s3": "^3.693.0",
    "@aws-sdk/s3-request-presigner": "^3.693.0",
    "@hookform/resolvers": "^5.2.2",
    "@radix-ui/react-accordion": "^1.2.12",
    "@radix-ui/react-alert-dialog": "^1.1.15",
    "@radix-ui/react-aspect-ratio": "^1.1.7",
    "@radix-ui/react-avatar": "^1.1.10",
    "@radix-ui/react-checkbox": "^1.3.3",
    "@radix-ui/react-collapsible": "^1.1.12",
    "@radix-ui/react-context-menu": "^2.2.16",
    "@radix-ui/react-dialog": "^1.1.15",
    "@radix-ui/react-dropdown-menu": "^2.1.16",
    "@radix-ui/react-hover-card": "^1.1.15",
    "@radix-ui/react-label": "^2.1.7",
    "@radix-ui/react-menubar": "^1.1.16",
    "@radix-ui/react-navigation-menu": "^1.2.14",
    "@radix-ui/react-popover": "^1.1.15",
    "@radix-ui/react-progress": "^1.1.7",
    "@radix-ui/react-radio-group": "^1.3.8",
    "@radix-ui/react-scroll-area": "^1.2.10",
    "@radix-ui/react-select": "^2.2.6",
    "@radix-ui/react-separator": "^1.1.7",
    "@radix-ui/react-slider": "^1.3.6",
    "@radix-ui/react-slot": "^1.2.3",
    "@radix-ui/react-switch": "^1.2.6",
    "@radix-ui/react-tabs": "^1.1.13",
```

```
"@radix-ui/react-toggle": "^1.1.10",
"@radix-ui/react-toggle-group": "^1.1.11",
"@radix-ui/react-tooltip": "^1.2.8",
"@tanstack/react-query": "^5.90.2",
"@trpc/client": "^11.6.0",
"@trpc/react-query": "^11.6.0",
"@trpc/server": "^11.6.0",
"axios": "^1.12.0",
"class-variance-authority": "^0.7.1",
"clsx": "^2.1.1",
"cmdk": "^1.1.1",
"cookie": "^1.0.2",
"date-fns": "^4.1.0",
"dotenv": "^17.2.2",
"drizzle-orm": "^0.44.5",
"embla-carousel-react": "^8.6.0",
"express": "^4.21.2",
"framer-motion": "^12.23.22",
"input-otp": "^1.4.2",
"jose": "6.1.0",
"lucide-react": "^0.453.0",
"mysql2": "^3.15.0",
"nanoid": "^5.1.5",
"next-themes": "^0.4.6",
"nodemailer": "^9.0.1",
"react": "^19.2.1",
"react-day-picker": "^9.11.1",
"react-dom": "^19.2.1",
"react-hook-form": "^7.64.0",
"react-resizable-panels": "^3.0.6",
"recharts": "^2.15.2",
"sonner": "^2.0.7",
"streamdown": "^1.4.0",
"superjson": "^1.13.3",
"tailwind-merge": "^3.3.1",
"tailwindcss-animate": "^1.0.7",
"vaul": "^1.1.2",
"wouter": "^3.3.5",
"zod": "^4.1.12"
},
"devDependencies": {
  "@builder.io/vite-plugin-jsx-loc": "^0.1.1",
  "@tailwindcss/typography": "^0.5.15",
  "@tailwindcss/vite": "^4.1.3",
```

```
"@types/express": "4.17.21",
"@types/google.maps": "^3.58.1",
"@types/node": "^24.7.0",
"@types/nodemailer": "^8.0.1",
"@types/react": "^19.2.1",
"@types/react-dom": "^19.2.1",
"@vitejs/plugin-react": "^5.0.4",
"add": "^2.0.6",
"autoprefixer": "^10.4.20",
"drizzle-kit": "^0.31.4",
"esbuild": "^0.25.0",
"pnpm": "^10.15.1",
"postcss": "^8.4.47",
"prettier": "^3.6.2",
"tailwindcss": "^4.1.14",
"tsx": "^4.19.1",
"tw-animate-css": "^1.4.0",
"typescript": "5.9.3",
"vite": "^7.1.7",
"vite-plugin-manus-runtime": "^0.0.58",
"vitest": "^2.1.4"
},
"packageManager": "pnpm@10.4.1+sha512.c753b6c3ad7afa13af388fa6d808035a0",
"pnpm": {
  "patchedDependencies": {
    "wouter@3.7.1": "patches/wouter@3.7.1.patch"
  },
  "overrides": {
    "tailwindcss>nanoid": "3.3.7"
  }
}
}
```

File: vite.config.ts

```
import { jsxLocPlugin } from "@builder.io/vite-plugin-jsx-loc";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import path from "node:path";
import { defineConfig } from "vite";

export default defineConfig({
  plugins: [react(), tailwindcss(), jsxLocPlugin()],
  resolve: {
    alias: {
      "@": path.resolve(import.meta.dirname, "client", "src"),
      "@shared": path.resolve(import.meta.dirname, "shared"),
      "@assets": path.resolve(import.meta.dirname, "attached_assets"),
    },
  },
  envDir: path.resolve(import.meta.dirname),
  root: path.resolve(import.meta.dirname, "client"),
  publicDir: path.resolve(import.meta.dirname, "client", "public"),
  build: {
    outDir: path.resolve(import.meta.dirname, "dist"),
    emptyOutDir: true,
  },
  server: {
    host: true,
    allowedHosts: ["localhost", "127.0.0.1"],
    fs: {
      strict: true,
      deny: ["**/*. *"],
    },
  },
});
```

File: vercel.json

```
{
  "version": 2,
  "buildCommand": "pnpm install && pnpm run build",
  "outputDirectory": "dist",
  "framework": "vite",
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```